

Raspberry Pi Camera Module Setup Guide

Introduction

This guide walks through the complete setup process for a Raspberry Pi with an IMX290/IMX462 camera module, including device tree overlay configuration, verification steps, and debugging techniques.

Prerequisites

- Raspberry Pi 4 or 5 with Raspberry Pi OS (Bookworm or later)
- IMX290 or IMX462 camera module with CSI-2 ribbon cable
- SSH access to the Raspberry Pi or direct terminal access
- Knowledge of basic command-line operations

Part 1: Initial Hardware Setup

Step 1: Connect the Camera Module

1. Power off the Raspberry Pi completely
2. Locate the Camera CSI-2 connector (between the Ethernet and USB-C ports on Pi 4, or near the IO headers on Pi 5)
3. Gently pull up the black retention clip on the CSI connector
4. Insert the ribbon cable with the blue side facing the Ethernet port (or toward the back of the board)
5. Push the retention clip down until it clicks
6. Power on the Raspberry Pi

Step 2: Enable I2C Interface

The camera communicates via I2C, which must be enabled:

```
sudo raspi-config
```

Navigate to: **Interface Options** → **I2C** → **Yes** → **Finish**

Or edit directly:

```
sudo nano /boot/firmware/config.txt
```

Ensure these lines are present (uncommented):

```
dtparam=i2c_arm=on  
dtparam=i2c_vc=on
```

Save: `Ctrl+X` → `Y` → `Enter`

Part 2: Configure Device Tree Overlay

The device tree overlay tells the kernel how to communicate with your specific camera model.

Step 3: Edit the config.txt File

```
sudo nano /boot/firmware/config.txt
```

For IMX290 Camera

Locate the camera auto-detection line and modify:

Find:

```
camera_auto_detect=1
```

Change to:

```
camera_auto_detect=0
```

Then add to the [all] section:

```
dtoverlay=imx290,clock-frequency=74250000,cam1  
gpu_mem=128
```

Parameters explained:

- `imx290` - Camera sensor type (use `imx462` if you have that model)
- `clock-frequency=74250000` - Clock frequency in Hz (74.25 MHz standard)
- `cam1` - Camera port (use `cam0` for single camera or alternative port)
- `gpu_mem=128` - GPU memory allocation (128 MB minimum for camera)

For IMX462 Camera

If using the IMX462 variant, the overlay is similar but the sensor name differs:

```
dtoverlay=imx462,clock-frequency=74250000,cam1  
gpu_mem=128
```

Step 4: Save and Reboot

After editing config.txt:

```
sudo reboot
```

Wait for the Raspberry Pi to restart (approximately 1-2 minutes).

Part 3: Verify Camera Installation

Step 5: Check Available I2C Buses

First, verify the I2C interfaces are loaded:

```
ls /dev/i2c*
```

Expected output (may vary):

```
/dev/i2c-1 /dev/i2c-20 /dev/i2c-21
```

Different buses represent different physical I2C connections. The actual camera I2C is internal (not directly numbered in /dev).

Step 6: Scan I2C Buses (Optional - See Debugging Section)

You can attempt to scan for I2C devices:

```
i2cdetect -y 20
```

```
i2cdetect -y 21
```

Note: On modern libcamera systems (Raspberry Pi OS Bookworm+), these buses may show unusual patterns because the GPU manages camera I2C directly. See Debugging section for explanation.

Step 7: List Available Cameras (MOST RELIABLE METHOD)

Use the libcamera utility to detect connected cameras:

```
rpicam-hello --list-cameras
```

Expected output for IMX290/IMX462: Available cameras

```
0 : imx462 [1920x1080 12-bit RGGB] (/base/soc/i2comux/i2c@1/imx290@1a)
Modes: 'SRGB10_CSI2P' : 1280x720 [60.00 fps - (320, 180)/1280x720 crop]
1920x1080 [60.00 fps - (0, 0)/1920x1080 crop]
'SRGB12_CSI2P' : 1280x720 [60.00 fps - (320, 180)/1280x720 crop]
1920x1080 [60.00 fps - (0, 0)/1920x1080 crop]
```

Success indicators:

- Camera model is listed (imx290 or imx462)
- I2C address shows as @1a (standard I2C address)
- Resolution and frame rates are available

- Multiple color modes (SRGGB10_CSI2P, SRGGB12_CSI2P) are supported

Step 8: Test Camera Functionality

Take a test image to verify the camera works:

```
rpicam-still -o test.jpg
```

Or preview for 5 seconds:

```
rpicam-hello --timeout 5000
```

Look for a preview window (if using desktop) or confirmation that the image was captured.

Part 4: Debugging Steps

Issue: Camera Not Detected

If `rpicam-hello --list-cameras` shows no cameras:

Check 1: Verify Hardware Connection

Look for any error messages in kernel logs

```
dmesg | grep -i camera  
dmesg | grep -i imx
```

If nothing appears, the kernel may not be detecting the camera hardware.

Check 2: Verify Device Tree Overlay Loaded

```
dtoverlay -l
```

Expected output should include:
imx290

If not listed, the overlay didn't load. Check config.txt for syntax errors.

Check 3: Verify GPIO and Power

Check if camera power pin is enabled

```
cat /proc/device-tree/base/soc/gpio@7e200000/camera_o__power/status
```

Should show "okay" if powered correctly.

Check 4: Verify I2C Communication

List all I2C buses and their drivers:

```
ls -la /dev/i2c*  
i2cdetect -l
```

The camera should be on one of the i2c buses (typically shown as i2c@1 internally).

Check 5: Review Full Kernel Logs

```
sudo journalctl -u libcamera-server -n 50
```

Or check for media framework messages:

```
dmesg | tail -20
```

Issue: i2cdetect Shows All Addresses Responding

This is normal on modern systems and not a failure condition.

Explanation

Buses like `/dev/i2c-20` and `/dev/i2c-21` may show all addresses responding when probed with `i2cdetect`. This occurs because:

1. **GPU controls camera I2C directly** - The Broadcom VideoCore GPU manages camera I2C communication, not the ARM processor
2. **Probing can interfere** - Direct I2C probing may cause the GPU to respond with all addresses
3. **Not a camera issue** - The camera is on an internal I2C bus (i2c@1) managed by the GPU, not on numbered buses like i2c-20

Solution: Ignore `i2cdetect` output for camera verification. Use `rpikam-hello --list-cameras` instead, which queries libcamera (the correct interface).

Issue: Camera Detected but Won't Capture

Check 1: Verify GPU Memory Allocation

```
grep gpu_mem /boot/firmware/config.txt
```

Should show:
`gpu_mem=128`

Or higher (256 for higher resolutions). Increase if needed and reboot.

Check 2: Check Camera Permissions

```
groups $USER
```

If "video" group is not listed:

```
sudo usermod -aG video $USER
```

Then log out and back in for changes to take effect.

Check 3: Test with libcamera-hello

```
libcamera-hello --timeout 5000
```

This is an alternative capture method that confirms libcamera functionality.

Issue: "Cannot open camera device"

Check 1: Verify Correct Camera Overlay

Ensure you're using the correct overlay name in config.txt:

- For IMX290: `dtoverlay=imx290`
- For IMX462: `dtoverlay=imx462`

Verify in config.txt:

```
grep dtoverlay /boot/firmware/config.txt
```

Check 2: Check for Duplicate or Conflicting Overlays

```
grep -i "dtoverlay.*cam" /boot/firmware/config.txt
```

Should only show one camera overlay. Remove duplicates or conflicting lines.

Check 3: Verify CSI-2 Connector Port

If using multiple cameras or if camera is connected to alternate CSI port, specify correctly:

```
dtoverlay=imx290,clock-frequency=74250000,cam0
```

(Change `cam1` to `cam0` or `cam1` based on physical port used)

Part 5: Advanced Configuration

Using Higher Resolution

For 4K capture (if sensor supports):

```
rpicas-still -o image.jpg --width 3840 --height 2160
```

Check supported resolutions:

```
rpicas-hello --list-cameras
```

Look for available modes in the output.

Adjusting Sensor Parameters

Exposure control

```
rpicam-still -o test.jpg --shutter 10000 --gain 2
```

White balance

```
rpicam-still -o test.jpg --awb daylight
```

Brightness/contrast

```
rpicam-still -o test.jpg --brightness 0.2 --contrast 1.5
```

Recording Video

Record 10 seconds of 1080p video at 30fps

```
rpicam-vid -o video.h264 --width 1920 --height 1080 --framerate 30 --timeout 10000
```

Troubleshooting Checklist

Use this checklist when the camera is not working:

- [] Hardware is securely connected (ribbon cable fully inserted)
- [] `sudo raspi-config` shows I2C enabled
- [] `/boot/firmware/config.txt` has correct `dtoverlay` line with camera model
- [] GPU memory is allocated (`gpu_mem=128` or higher)
- [] System has been rebooted since config changes
- [] `rpicam-hello --list-cameras` shows camera detected
- [] Camera address shows as `@1a`
- [] Sensor is listed as `imx290` or `imx462`
- [] Resolutions and frame rates are available
- [] User is in video group (`groups $USER` includes "video")
- [] `rpicam-hello --timeout 5000` runs without errors

Summary of Key Commands

Enable I2C (one-time)

```
sudo raspi-config
```

Edit configuration

```
sudo nano /boot/firmware/config.txt
```

Reboot to apply changes

```
sudo reboot
```

List available cameras (primary verification)

```
rpicam-hello --list-cameras
```

Check loaded device tree overlays

```
dtoverlay -l
```

Scan I2C buses (supplementary)

```
i2cdetect -y 20  
i2cdetect -y 21
```

View kernel logs for camera

```
dmesg | grep -i camera
```

Capture test image

```
rpicam-still -o test.jpg
```

Preview for 5 seconds

```
rpicam-hello --timeout 5000
```

Record video

```
rpicam-vid -o video.h264 --timeout 10000
```

Part 6: Virtual Environment and Repository Setup

This section installs the Python tools used by the project, confirms I2C is enabled, creates a virtual environment, and downloads the project source code.

Step 1: Install Necessary Tools

```
sudo apt update
```

```
sudo apt install -y python3-pip python3-venv python3-smbus i2c-tools python3-libcamera  
python3-picamera2
```

These packages provide pip, virtual environment support, I2C utilities, SMBus access, and the Raspberry Pi camera libraries used by the application.

Step 2: Make Sure I2C Is Enabled

```
sudo raspi-config
```

Navigate to Interface Options -> I2C -> Enable

If I2C was already enabled earlier in the setup, this step serves as a quick confirmation before installing and running the sensor-dependent Python code.

Step 3: Create and Activate the Virtual Environment

```
python3 -m venv --system-site-packages .venv
```

```
source .venv/bin/activate
```

Using `--system-site-packages` allows the virtual environment to access Raspberry Pi system packages such as camera-related libraries installed through `apt`.

Step 4: Install Python Packages for Project Components

```
pip install --upgrade pip
```

```
pip install adafruit-blinka
```

```
pip install adafruit-circuitpython-tsl2591
```

```
pip install adafruit-circuitpython-seesaw
```

```
pip install pygame
```

These packages support the hardware and interface portions of the project, including Adafruit CircuitPython device access, the TSL2591 light sensor, Seesaw-based components, and pygame dependencies.

Step 5: Clone the Project Repository

```
git clone https://github.com/albaparsi/Camera-Project-Source.git
```

```
cd Camera-Project-Source
```

The `git clone` command downloads the full project source into a new folder named `Camera-Project-Source`. After changing into that directory, the files can be reviewed, configured, and run from inside the activated virtual environment.

Step 6: Recommended Startup Order

Open a terminal in the project directory.

Activate the virtual environment with `source .venv/bin/activate` if it is not already active.

Verify that the camera is still detected with `picam-hello --list-cameras`.

Run the project-specific Python scripts or launcher described in the repository files.

If the repository later includes a `requirements` file or newer run instructions, those repository-specific steps should take priority over older local notes.

References

[1] Raspberry Pi Foundation. (2024). Camera Module Configuration. <https://www.raspberrypi.com/documentation/accessories/camera.html>

[2] Libcamera Project. (2024). Raspberry Pi Camera Support. <https://libcamera.org/>

[3] Broadcom VideoCore GPU Documentation. Device Tree Bindings for IMX290 Sensor.